

Ripasso rapido — Lezione 14

HTML + JS

1. Collegare JS all'HTML

- Lo script va collegato con `<script src="...">` in fondo al `<body>`, appena prima di `</body>`.
- Così, quando lo script parte, tutto l'HTML sopra è già stato letto dal browser.

```
<body>
  <h1>Pagina</h1>
  <script src="script.js"></script>
</body>
```

2. querySelector e l'id come gancio

- L'id in HTML è un "gancio": `document.querySelector("#id")` prende esattamente quell'elemento.
- Si salva il risultato in una costante, una volta sola, in cima al file.

```
const btn = document.querySelector("#btnAggiungi");
```

3. .value dentro l'onclick (non fuori)

- Il valore di un input va letto al momento del click, dentro la funzione collegata all'evento.
- Letto fuori (in cima al file) prende sempre il valore iniziale, vuoto.

```
btn.onclick = () => {
  const nome = input.value; // letto ORA, non prima
};
```

4. innerHTML vs innerText

- `innerHTML` interpreta il testo come HTML (permette tag come `<strong>`).
- `innerText` scrive solo testo puro, anche se contiene simboli `<` e `>`.

```
div.innerHTML = "<strong>Ciao</strong>"; // grassetto vero
div.innerText = "<strong>Ciao</strong>"; // scritto letteralmente
```

5. Pattern EAR — Evento -> Azione -> Reazione

- Evento: qualcosa succede (click, submit, input).
- Azione: i dati (l'array/variabile di stato) vengono aggiornati.
- Reazione: la pagina viene ridisegnata di conseguenza.

```
btn.onclick = () => {           // Evento
  lista.push(input.value);      // Azione
  render();                     // Reazione
};
```

6. Pattern render()

- L'array (o variabile) in JS è la fonte di verità, l'HTML è solo la sua fotografia.
- `render()` ricostruisce da zero il pezzo di pagina leggendo l'array.

```
let lista = [];

function render() {
```

```

    ul.innerHTML = "";
    lista.forEach((el) => {
        ul.innerHTML += `<li>${el}</li>`;
    });
}

```

7. template.replace() per costruire HTML

- Si scrive un "modello" di stringa con un segnaposto e lo si sostituisce col valore vero.
- Utile per righe di tabella, card, messaggi ripetuti.

```

const template = "<li>Nome: %NOME</li>";
const riga = template.replace("%NOME", "Marco");

```

Array e Dizionari

8. I metodi array piu usati

- `forEach`: esegue un'azione per ogni elemento, non restituisce nulla.
- `map`: trasforma ogni elemento, restituisce un nuovo array della stessa lunghezza.
- `filter`: restituisce un nuovo array piu corto, solo elementi che rispettano una condizione.
- `find`: come filter ma si ferma al primo trovato, restituisce l'elemento (non un array).
- `sort`: riordina l'array (modifica l'originale), usare $(a,b) \Rightarrow a-b$ oppure $(a,b) \Rightarrow b-a$.
- `reduce`: riduce l'array a un unico valore accumulato (somma, media, max...).

```

const numeri = [3, 8, 1, 5];
numeri.forEach((n) => console.log(n));
const doppi = numeri.map((n) => n * 2);
const grandi = numeri.filter((n) => n > 3);
const primo = numeri.find((n) => n > 3);
numeri.sort((a, b) => a - b);
const somma = numeri.reduce((acc, n) => acc + n, 0);

```

9. for...in per i dizionari

- Scorre le chiavi di un oggetto (non i valori).

```

const studente = { nome: "Luca", voto: 7 };
for (const chiave in studente) {
    console.log(chiave, studente[chave]);
}

```

10. Object.keys() e Object.values()

- `Object.keys(obj)` restituisce l'array delle chiavi.
- `Object.values(obj)` restituisce l'array dei valori.

```

Object.keys(studente); // ["nome", "voto"]
Object.values(studente); // ["Luca", 7]

```

CSV

11. Le 4 righe fondamentali del parsing CSV

- `shift()` rimuove E restituisce il primo elemento dell'array (modifica l'originale): serve per l'header.

- I valori letti da un CSV sono sempre stringhe: vanno convertiti con `Number()` prima di calcoli/confronti.

```
let testo = csv.trim(); // 1. pulisco spazi/newline ai bordi
testo = testo.replaceAll(" ", ""); // 2. tolgo spazi interni
const righe = testo.split("\n"); // 3. divido per riga
const header = righe.shift().split(","); // 4. tolgo e divido l'header
```

12. Costruire array di dizionari dal CSV

- Per ogni riga si crea un dizionario associando ogni valore alla chiave dell'header nella stessa posizione.

```
const studenti = righe.map((riga) => {
  const valori = riga.split(",");
  const obj = {};
  header.forEach((chiave, i) => {
    obj[chiave] = valori[i];
  });
  return obj;
});
```

13. Generare CSV risultato con map e join

- `map()` trasforma ogni dizionario in una riga di testo, `join()` unisce le righe con l'a-capo.

```
const righeOutput = studenti.map((s) => Object.values(s).join(","));
const csvOutput = [header.join(","), ...righeOutput].join("\n");
```

AJAX e Fetch

14. Schema base fetch

- Sempre due `await`: uno per la richiesta (`fetch`), uno per leggerne il contenuto (`.json()`).
- Regola d'oro: `console.log(data)` SEMPRE, per capire la struttura prima di usarla.

```
const response = await fetch(url); // 1 await -> Response
const data = await response.json(); // 2 await -> dati veri
console.log(data);
```

15. GET vs POST

- GET: si usa per leggere/chiedere dati, basta `fetch(url)`, niente `method/body`.
- POST: si usa per mandare dati al server, servono `method "POST"`, `headers` e `body`.
- `body` è sempre una stringa: si costruisce con `JSON.stringify({...})`.

```
// GET
const r1 = await fetch(url);

// POST
const r2 = await fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ nome: "Mario" }),
});
```

16. URL template con .replace()

- Si scrive un URL modello con un segnaposto e lo si sostituisce col valore vero prima della `fetch`.
- Con più parametri si concatenano più `.replace()`.

```
const url = "https://api.open-meteo.com/v1/forecast?latitude=%LAT&longitude=%LON&current_weather=true";
url.replace("%LAT", lat)
url.replace("%LON", lon);
```

17. Fetch POST: struttura

- method: "POST" + headers (Content-Type application/json) + body (JSON.stringify).

```
const response = await fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ nome: "Mario", voto: 7 }),
});
```

Cache remota

18. Cache remota — SET

- Il TOKEN (personale) va nell'header "key"; la chiave del dato va nel body: non vanno confusi.
- Il value va sempre passato a JSON.stringify prima di salvarlo.

```
await fetch("https://ws.cipiacinfo.it/cache/set", {
  method: "POST",
  headers: { "Content-Type": "application/json", "key": TOKEN },
  body: JSON.stringify({
    key: "nomeChiave",
    value: JSON.stringify(valore),
  }),
});
```

19. Cache remota — GET

- data.result è una STRINGA se la chiave esiste, un OGGETTO se non esiste: si controlla con typeof.

```
const data = await response.json();
if (typeof data.result === "string") {
  const valore = JSON.parse(data.result);
} else {
  // chiave non trovata
}
```

20. JSON.stringify prima di salvare, JSON.parse dopo aver letto

- Il value salvato deve essere una stringa: JSON.stringify(valore) prima del SET.
- data.result tornato dal GET è ancora una stringa da riaprire: JSON.parse(data.result) dopo.
- Vale anche per array/oggetti interi (es. una lista intera salvata in un colpo solo).

```
// salvo
value: JSON.stringify(valore)

// leggo
const valore = JSON.parse(data.result);
```